

Automation of Incident Management with AI

Suppose AI creating Root cause analysis report upon sharing the metrics and logs to it on its own.

Contains:

- **The Code Files:** Simple text files (usually ending in .py for Python) that tell the AI what to do.
- **The Requirements File (requirements.txt):** A shopping list of free software tools your project needs to run.
- **The README (README.md):** A beautifully formatted text document that explains to anyone visiting your GitHub exactly what your project does and how to use it. This is what hiring managers look at first.

The Concept

Instead of an engineer spending two hours digging through logs and metrics to write a post-mortem report after an outage, a user pastes the error logs and system metrics into a simple web interface, and the AI outputs a professional RCA document instantly.

Real-Time Example

The Scenario: A critical payment gateway goes down.

The Input: You feed the AI a few lines of server logs showing HTTP 504 Gateway Timeout and Connection pool exhausted, alongside a metric note that CPU utilization hit 99%.

The AI Action: The AI reads this data, deduces that a sudden traffic spike caused the database connections to lock up, and automatically generates a structured text file containing the Incident Timeline, Root Cause, and Recommended Actions.

Step-by-Step Execution Plan

Step 1: We will create a simple, free web layout using a Python tool called Streamlit (it lets you build web screens using plain English logic). It will have a text box labeled "Paste Logs Here."

Step 2: We connect that text box to an AI model (like OpenAI's GPT-4o-mini). We give the AI a strict rule: "You are a Senior SRE. Read these logs and fill out this exact RCA template."

Step 3: The screen displays a "Download RCA Report" button for the engineer.

Step 1 Requirements

```
streamlit==1.32.0
openai==1.14.0
```

Why this file matters: streamlit builds our web screen, and openai lets our code talk to the AI brain.

Step 2: Create the App Logic (app.py)

Now for the exciting part—the actual code. Don't worry, I've commented on every single line to explain exactly what it does.

Click Add file → Create new file in github. Name the file exactly: `app.py`. Paste the following block of code into the Github file: The Code is available in ap.py file/folder in github.

Action steps taken to see how it actually performed

1. Install python
2. Open Terminal in mac
3. Paste the below command in terminal: `pip install streamlit openai`

The 3rd action step was unable to run the command in terminal and give us error —this happens to almost everyone when they first set up Python on their machine. Usually, it just means your terminal doesn't know exactly where to look for the pip command yet. Then, I used this command: `python3 -m pip install streamlit openai` - It worked out fine for me.

4. Create the File Locally

Create a new folder on your computer named `ai-rca-generator`. Open a text editor (like Notepad, TextEdit, or VS Code), copy the full app.py code from our previous conversation, and save it inside that folder as app.py.

While performing i got error and was unable to complete the action. Mac has a sneaky habit of trying to turn everything into a Rich Text file (.rtf) or adding a hidden .txt to the end of your filename (like app.py.txt). When that happens, Python gets confused because it's looking for a pure code file. Since I already have the Terminal open, let's bypass the desktop apps completely and create the file directly from your Terminal. It works 100% of the time and is much faster.

The Foolproof Terminal Method Sub Steps:

- **1. Navigate to your folder:** Type `cd` (with a space after it) in Terminal, then drag and drop ai-rca-generator folder from our desktop right into the Terminal window. Hit Enter. (our terminal line should now show that we are inside that folder).
- **2. Open the Terminal's built-in text editor:** Type this command and hit Enter: `nano app.py`. A black screen will open inside your terminal. This is nano, a simple text editor built into every Mac.
- **3. Paste the code:** Copy the full Python code block for app.py from our earlier conversation. Click inside the Terminal window and press `Cmd + V` to paste it. we will see the code fill up the screen.
- **4. Save and Exit:** Look at the shortcuts at the bottom of your terminal screen (the ^ symbol means the Control key, not Command): Press `Control + O` (this means "WriteOut" or save). Hit Enter to confirm the filename is app.py. Press `Control + X` to exit the editor and go back to your normal terminal screen. Type `ls` and hit Enter. We should see app.py sitting there perfectly!

5. Once done we need to run the app - `streamlit run app.py` command in terminal - Sadly, it didnt wroked out for me - This is another classic Mac quirk! What is happening here is that Python successfully downloaded Streamlit, but our Mac hasn't added the "Streamlit" shortcut command to its main dictionary yet. The easiest way to bypass this without messing with your Mac's internal system paths is to tell Python to launch Streamlit directly. Instead of running `streamlit run app.py` , copy and paste this exact command into your terminal and hit Enter: `python3 -m streamlit run app.py`

Why this works: By adding `python3 -m`, we are telling our computer: "Hey Python, go into your own internal toolbox, find the 'streamlit' package I downloaded earlier, and use it to run my `app.py` file."

And now it worked, the interface window opened in browser, (<http://localhost:8501/>) Take a second to realize what we just accomplished: troubleshooting packages, handled file syntax errors, manipulated hidden text files through the command line via the terminal, and successfully stood up a local web application server. That is core IT engineering work!

Now coming to the last step of pasting the logs in browser and downloading the RCA report, but again When i open the browser to check the RCA report - its asking me for Open API key - Because we wrote this line in your code: `api_key = st.sidebar.text_input("Enter your OpenAI API Key")` , your app is securely waiting for your personal access pass to talk to the AI brain.

Instead, let's use a 100% free cloud alternative that requires zero credit cards, zero deposits, and takes 1 minute to set up. We are going to use Google AI Studio. Google gives developers a generous free tier to use their Gemini models without charging a cent. The best part? Google's AI service can pretend to be OpenAI, which means we don't have to install any new toolboxes. We just change the destination address in your `app.py` file.

Step 1: Get our Free Google Gemini Key

- Go to Google AI Studio and log in with your normal Google account.
- Click the prominent blue button in the top left: "Get API key".
- Click "Create API key", select a project (or create a default one), and copy the long key it gives you.

Step 2: Update our `app.py` Code

- Let's open your terminal text editor and point your app to Google's free servers.
- In your terminal, make sure you stopped the server (Control + C).
- Open the file: `nano app.py`
- Hold Control + K until the screen is completely blank.
- Paste this updated, Gemini-compatible code into the file.

Finally, It worked and I got the results -

AI Root Cause Analysis (RCA) Generator

Paste your raw logs and system metrics below to instantly generate a professional SRE Incident Report.

Incident Title / Paste System Logs / Metrics Here

RCA Report Generated Successfully!

Incident Root Cause Analysis (RCA)

Incident Reference: INC-2026-06-30-PAY | **Date of Report:** 2026-06-30

Severity: P1 - Critical | **Service Impacted:** Payment Gateway / auth-service-v2

Author: Principal Site Reliability Engineer

1. Executive Summary

On June 30, 2026, at approximately 10:14:40 UTC, the Payment Gateway began returning 502 Bad Gateway errors to clients attempting to process transactions. The underlying cause was identified as the sudden termination of the auth-service-v2 dependency. The service, running a Python-based application inside a Kubernetes cluster, experienced a rapid increase in memory consumption immediately after startup at 10:00:00 UTC. By 10:14:32 UTC, the container exceeded its configured memory limit of 512MB, triggering the Linux Kernel Out-Of-Memory (OOM) killer (CONSTRAINT_MEMCG).

Kubernetes terminated the pod with Exit Code 137 (OOMKilled). Because the authentication service was offline, the Nginx Ingress/Gateway could not route authentication requests, resulting in HTTP 502 errors. Service was restored once Kubernetes restarted the pod and traffic was successfully re-routed, though the underlying memory profile issue requires immediate remediation.

2. Incident Timeline Hypothesis

Timestamp (UTC)	Source	Event / Metric	SRE Analysis & Impact
10:00:00Z	auth-service-v2	[INFO] auth-service-v2 starting up...	Service initialized. Initial memory footprint is already high or rapidly allocating resources.
10:05:00Z	Metrics	container_memory_working_set_bytes=480MB	Memory usage is at 93.7% of the 512MB threshold within 5 minutes of boot.
10:12:00Z	Metrics	container_memory_working_set_bytes=509MB	Memory usage reaches 99.4% of the threshold. The container is in a critical state with no headroom for spike traffic.
10:14:32Z	Kernel	oom-kill: constraint=CONSTRAINT_MEMCG, task=python3, pid=412	The Linux kernel invokes the OOM killer inside the memory control group (cgroup) to protect the host node, terminating the main Python process (PID 412).

Timestamp (UTC)	Source	Event / Metric	SRE Analysis & Impact
10:14:35Z	Kubernetes	Pod auth-service-v2-7df789b-x2k9s terminated with Exit Code 137	Kubernetes detects the hard termination (SIGKILL / Exit Code 137) due to OOM.
10:14:40Z	Nginx Gateway	[WARN] Nginx Gateway: 502 Bad Gateway	User-facing impact begins. Nginx cannot establish a connection to the upstream auth-service-v2 backend, returning 502 Bad Gateway errors for payment requests.

3. Root Cause Determination

The root cause of the payment gateway failure was a cascading failure triggered by resource starvation in the upstream authentication service (auth-service-v2).

```
[Memory Leak / High Baseline]
    €<
    â-¼ (Reaches 512MB Limit)
[Kernel OOM-Killer (PID 412)]
    €<
    â-¼ (Exit Code 137)
[Pod Termination]
    €<
    â-¼ (No Upstream Available)
[Nginx Gateway 502 Bad Gateway]
```

Technical Breakdown:

- **Insufficient Resource Limits / Memory Leak:** The auth-service-v2 container was configured with a hard memory limit of 512MB. Within 5 minutes of startup, the working set memory was already at 480MB. This indicates either: An inappropriately low memory limit configured in the Kubernetes deployment manifest for the workload size, or a severe memory leak or massive initialization cache load in the Python application (python3, PID 412).
- **Cgroup Enforcement:** At 10:14:32 UTC, the container memory usage crossed the 512MB threshold. The cgroup engine enforced the limit (CONSTRAINT_MEMCG) and sent a SIGKILL (9) signal to the primary process.
- **Gateway Ingress Failure:** When the pod was terminated, the Kubernetes endpoint controller removed the pod IP from the active endpoints. However, during this propagation window, or due to lack of healthy replicas, the Nginx Gateway had no valid upstream servers to route authentication requests to, resulting in immediate 502 Bad Gateway responses.

4. Preventative Actions

To prevent recurrence of this issue, the following short-term and long-term actions have been defined:

Short-Term Mitigations (Immediate)

- **Action 1: Increase Memory Limits (Hotfix):** Immediately increase the Kubernetes memory limit for auth-service-v2 from 512Mi to 1Gi (or 2Gi depending on staging tests) to provide immediate operational headroom.
- **Action 2: Scale Replicas:** Ensure the deployment has a minimum of 3 replicas spread across different nodes to ensure high availability and prevent a single pod crash from causing a complete gateway outage.
- **Action 3: Configure Liveness/Readiness Probes:** Verify and tune the HTTP readiness and liveness probes for auth-service-v2 to ensure Nginx quickly stops routing traffic to failing/unresponsive pods before they are OOM-killed.

Long-Term Preventative Actions (Strategic)

- **Action 4: Python Memory Profiling:** Task the backend engineering team to profile the auth-service-v2 Python application (using tools like tracemalloc or memory_profiler) to identify why the startup memory footprint is abnormally high (~480MB) and check for memory leaks.
- **Action 5: Implement Prometheus Alerting:** Set up Prometheus/Alertmanager alerts to trigger when container memory utilization exceeds 80% of its limit for more than 2 minutes (e.g., $\text{container_memory_working_set_bytes} / \text{container_spec_memory_limit_bytes} > 0.80$). This will allow SREs to intervene before an OOM-kill occurs.
- **Action 6: Horizontal Pod Autoscaler (HPA):** Implement an HPA policy based on memory utilization to automatically scale out replicas when load or memory consumption spikes.